北京大学信息科学技术学院考试试卷

数据结构与算法 A 2021 年期中试卷（整理版）

考试科目：数据结构与算法 A 姓名：_____ 学号：_____

考试年份：2021 年 任课教师：_____

题号	一	二	三	四	总分
得分					
阅卷人					

整理说明：依据“21年期中”文件夹 20 张图片重排，参照 2022 年试卷的卷首、题型分区和页码样式。原图未见各题分值；二、3 与二、4 未提供，三、1 代码中段缺失，均在原位注明。保留原题号。

答题说明：请将答案写在题目空白处或另附答题纸。部分选择题为多选，依题意作答。参考答案统一附后。

一、选择填空

1. 以下算法的时间复杂度是 _____。

```
int m = 1, i, j;
for (i = 1; i <= n; i++)
    for (j = 1; j <= i; j *= 2)
        m *= 2;
```

A. $\Theta(\log_2 n)$ B. $\Theta(n)$ C. $\Theta(n\log_2 n)$ D. $\Theta(n^2)$

2. 在 _____ 中，即使丢失了头结点，只要指出表中任何一个结点的指针，也可以访问到该链表的所有结点。

A. 线性单链表 B. 双向链表 C. 线性链表 D. 循环链表

3. 采用栈将中缀表达式转换为后缀表达式的过程中，栈中不可能出现的状态是 _____？（左侧为栈底）

A. +/+ B. +(+ C. (+) D. +*(*+

一、选择填空（续）

4. 双端队列是指插入和删除操作限制在两端进行的线性表。若将 n 个互不相同的元素依次插入到双端队列中（仅插入不删除），则可得到 _____ 种不同的排列。

A. 2 B. 2n C. 2^{n-1} D. 2^n

5. 设顺序存储的循环队列 Q(1:40)，数组下标为 1 到 40，其中 front 指向队列头元素的位置，rear 指向队列尾元素的下一位置。设初始状态为 front=rear=40，经过一系列入队与出队运算后，front=20，rear=15。现要在该循环队列中寻找最小值的元素，最坏情况下需要比较的次数为 _____。

6. 若一棵二叉树的后序遍历序列是 {1, 3, 2, 6, 5, 7, 4}，中序遍历序列是 {1, 2, 3, 4, 5, 6, 7}，则下列哪句是错的 _____?

A. 2 是 1 和 3 的父结点

B. 7 是 5 的父结点

C. 这是一棵二叉搜索树

D. 这是一棵完全二叉树

7. 将森林 F 转换为对应的二叉树 T，F 中的叶结点的个数为 _____。

A. T 中叶结点的个数

B. T 中度为 1 的结点个数

C. T 中左子指针为空的结点个数

D. T 中右子指针为空的结点个数

8. 设一棵完全二叉树具有 1000 个结点，则此完全二叉树有 _____ 个叶子结点，有 _____ 个度为 2 的结点，有 _____ 个结点只有非空左子树，有 _____ 个结点只有非空右子树。

二、辨析与简答

1. 对于 t="aabaabaabaac" 的待匹配文本及 p="aabaac" 的模式，请求出 p 的优化后的 next 数组并进行 KMP 快速模式匹配，画出匹配过程的示意图。

2. 序列 23, 17, 14, 6, 13, 10, 1, 5, 8, 12 是否为一个最大值堆？若是，请说明理由；否则请严格按照筛选法建堆的过程将其调整成为最大值堆，并画出调整建堆的逐步过程。

3、4. 【原文件夹未提供这两题的图片，保留题号。】

5. 使用重量权衡合并规则与路径压缩优化，对下列各对元素进行合并。初始情况下，集合中的每个元素分别在独立的等价类中。合并时将结点少的树并入结点多的树；若两树规模相同，则把根值较大的并入根值较小的树。

(0,2) (1,2) (3,4) (3,1) (3,5) (9,11) (12,14) (3,9)
(4,14) (6,7) (8,10) (8,7) (7,0) (10,15) (10,13)

请填写所有等价对都被处理后，各结点的父结点下标。

下标	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
父结点																

三、算法填空

1. 对于一个由小写字母组成的字符串，我们可以多次采用数字和字母的组合来表示，用 x[s] 表示字符串 s 重复 x 次，其中 s 为任意长度的非空字符串，x 为正整数（0<x<300）。现给一段压缩后的字符串，请完善程序以输出解压后的字符串。

保证输入合法，且解压后的字符串长度不超过 100000。
样例输入：3[d2[c]] 样例输出：dccdccdcc

```
#include <bits/stdc++.h>
using namespace std;
const int N = 1e5 + 5;
char s[N], stk[N], t[N];
// stk: 栈数组；stk[1] 为栈底，stk[top] 为栈顶。
// top = 0 表示栈为空。
```

【原图缺失：上述声明与下方代码之间的代码页未提供，填空 1～3 的位置未见。下方保留现存续页及填空 4；卷末照录已有四项答案。】

```
                while (isdigit(stk[digi_pos])) {
                    _____________________;  // 填空 4
                    digi_pos++;
                }
                for (int j = 1; j <= num; j++)
                    for (int k = 1; k <= tn; k++) {
                        stk[lt] = t[k];
                        lt++;
                    }
                top = lt - 1;
            }
            else {
                top++;
                stk[top] = s[i];
            }
        }
        for (int i = 1; i <= top; i++)
            printf("%c", stk[i]);
        printf("\n");
        return 0;
    }
```

整理注：代码中的中文注释为说明性重排；缺失中段未猜补。

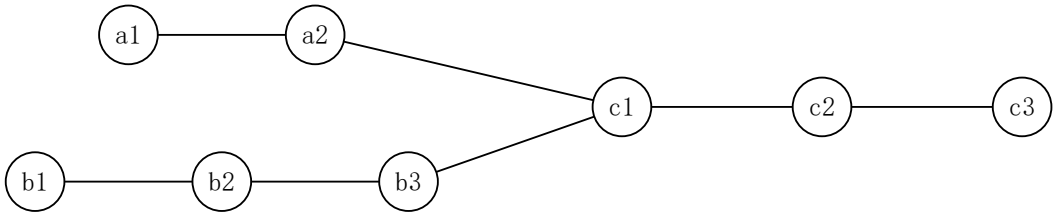
三、算法填空（续）

2. 二叉搜索树的存储情况如下，请补全代码，实现利用非递归方法找到二叉搜索树中第 k 大的数的功能。

```
class Node {
public:
    int val;
    Node *left, *right, *parent;
    bool visited;
    Node(int v) {
        val = v;
        parent = left = right = NULL;
        visited = false;
    }
    Node(int v, Node *l, Node *r, Node *p) {
        val = v; left = l; right = r; parent = p;
        visited = false;
    }
};
class BinaryTree {
public:
    Node *root;
    stack<Node *> st;
    BinaryTree(Node *r) { root = r; }
    BinaryTree() {}
    ~BinaryTree() { /* ... */ }
    // ...
    Node *kth_number_non_recursive(int k, Node *root) {
        while (root != NULL) {
            while (_____________________________) {  // 填空 5
                _____________________________;    // 填空 6
            }
            if (!root->visited) {
                _____________________________;    // 填空 7
                if (k == 0) break;
                root->visited = true;
            }
            if (_____________________________) {  // 填空 8
                _____________________________;    // 填空 9
            }
            else {
                root = root->parent;
            }
        }
        return root;
    }
};
```

四、算法设计与实现

1. 假设有两个单链表 A 和 B 有交叉结点（如图所示，但整个链表结构中不存在环），单链表 A 和 B 的头结点地址分别为 headA 和 headB。请设计算法，找出并返回这两个单链表相交的起始结点。请给出算法描述及其伪代码。算法要求满足 O(len(A)+len(B)) 的时间复杂度和 O(1) 的空间复杂度。若不满足复杂度要求，会酌情扣分。



2. 对于一个长度为 n 的字符串 S，我们称 S[0..i-1] 为字符串 S 长度为 i（1≤i≤n）的前缀。譬如字符串 acdf 的所有前缀是 a、ac、acd、acdf。现给出字符串 S，请你设计一个复杂度尽量低的算法来计算出 S 的所有前缀中，前缀长度与前缀出现次数的乘积的最大值。要求写出算法的思想，并分析时间复杂度。

例如 S="cdcdcdc"：前缀 c、cd、cdc、cdcd、cdcdc、cdcdcd、cdcdcdc 的（长度，出现次数，乘积）依次为 (1,4,4)、(2,3,6)、(3,3,9)、(4,2,8)、(5,2,10)、(6,1,6)、(7,1,7)。最大值为 10。

2021 年期中试卷 · 参考答案

说明：注明“原附答案”的内容来自图片；注明“整理补充”的内容为整理时计算或说明，便于核对。缺失原题与缺失答案分别标注。

一、选择填空

题号	1	2	3	4	5	6	7	8
答案	C	BD	ACD	C	34	D	C	500；499；1；0

1～5、7 为原附答案；6、8 为整理补充。

1. 内层执行次数为 floor($\log_2 i$)+1，总和为 $\Theta(n\log_2 n)$。
2. 双向链表可以先向前找到表头；循环链表可沿环访问全部结点。
3. 原附解析：栈中不可能出现相互匹配的一对括号；优先级低的运算符不可能紧接着优先级较高的运算符入栈。
4. 第一个元素插在左端或右端无区别，之后每个元素有两种插入方式，故为 2^{n-1}。
5. 元素个数为 15+(40-20)=35，求最小值需 35-1=34 次比较。

6. 【整理补充】根为 4，左子树根为 2，右子树根为 7；7 的左孩子为 5，而 5 的右孩子为 6，故不是完全二叉树。
8. 【整理补充】下标 1～500 为非叶结点；1～499 有两个孩子，第 500 个结点仅有左孩子。

二、辨析与简答

1. 【原附答案】优化 next 数组（下标从 0 开始）：

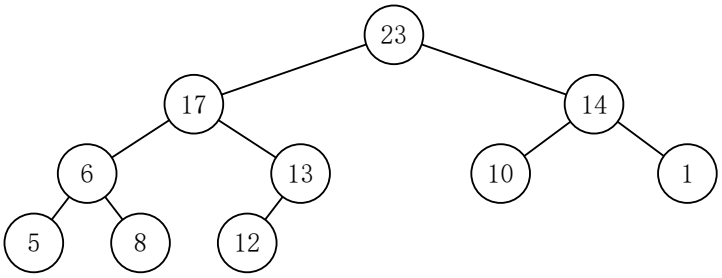
下标	0	1	2	3	4	5
p	a	a	b	a	a	c
next	-1	-1	1	-1	-1	2

```
文本：a a b a a b a a b a a c
模式：a a b a a c              (i=5, j=5 失配)
        a a b a a c            (i=8, j=5 失配)
          a a b a a c          (匹配成功，起始下标 6)
```

图示中每次 j=5 失配后按 next[5]=2 回退，文本指针不回退。

2021 年参考答案（续）

二、2. 【原附答案】否。结点 6 比其孩子 8 要小，所在子树不满足最大值堆的性质。原图展示的是待调整树形，重绘如下。



二、2【整理补充】自底向上筛选：
初始：[23,17,14,6,13,10,1,5,8,12]；
处理值为 6 的结点，与较大孩子 8 交换后得到
[23,17,14,8,13,10,1,5,6,12]；其余结点无需调整。

二、3、4：原图缺失。二、5：原文件夹未附答案。

二、5【整理补充】按每次合并时对两个查找路径作完整压缩、合并结束后不再额外全表压缩，所得父结点下标如下（根指向自身）：

下标	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
父结点	0	0	0	0	0	0	0	6	0	0	0	9	0	0	12	0

三、算法填空

1. 【原附答案，依次对应填空 1～4】

```
return c >= '0' && c <= '9';
stk[0] = ']';
t[j--] = stk[lt];
num = num * 10 + stk[digi_pos] - '0';
```

填空 1～3 所在代码页未提供，故只能保留答案，不能保证独立复现整段原程序。

2. 【原附答案，依次对应填空 5～9】

```
root->right != NULL && !root->right->visited
root = root->right;
k -= 1;
root->left != NULL && !root->left->visited
root = root->left;
```

原附说明：条件的前后顺序不能交换；k -= 1 可用同义语句替代。

四、算法设计与实现

1. 原附图片仅给出相交链表的示意图，交点为 c1，未附算法代码。

【整理补充】双指针分别从 headA 与 headB 出发，每次各走一步；到达空指针后转到另一条链表的头结点，直到两指针相等。两者分别走过 A+B 与 B+A 的等长路程，在首个公共结点相遇；无交点时同时为空。时间 O(len(A)+len(B))，额外空间 O(1)。

2. 原文件夹未附本题参考答案；本整理版保留题干及完整示例。